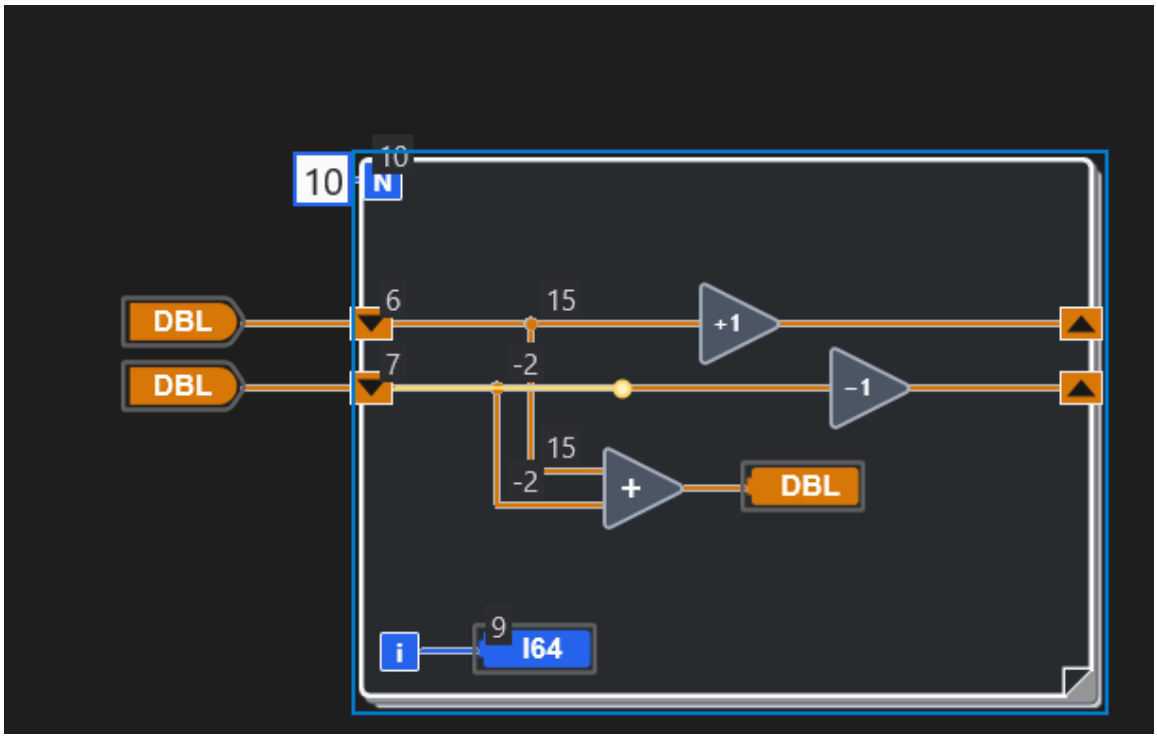
Graiphic Studio

Dataflow programming for the AI era

Human control of generated software
from inspection to executable results



Whitepaper

Version 1.0 • October 2026

Human in the Loop • Trustworthy AI • Inspectability

A continuation of the FROG foundation whitepaper

GRAIPHIC

Edition information

Version	Date	Edition status
1.0	24 October 2026	First English edition.

Preface

The FROG foundation whitepaper introduced an open graphical language for a new era of software creation. Its starting point was already ambitious: give humans and machines a shared, explicit representation of a program, independent of a particular editor or execution platform. Inspectability, provenance and AI assisted authoring belonged to that foundation. [1]

This whitepaper takes the next step. It explains how Graiphic Studio develops that foundation into a solution for understanding, improving and deploying software in an age of increasingly automated code production. The emphasis moves from the language alone to the engineer’s complete experience: create, inspect, simulate, debug, correct and deliver a working system.

We believe the graphical interface has a new strategic purpose. It remains a way to create software. It also becomes an interface through which people can exercise informed control over software produced with AI. The graphical dataflow programming language is central to this purpose: it gives inspection, experimentation and runtime understanding a shared structure.

FROG is the open language foundation. Graiphic Studio is a proprietary solution developed by Graiphic. This document presents our technology direction, product principles and intended business model. Release documentation and commercial terms will define the capabilities and guarantees available for each supported configuration. Public references support the external facts; our forecasts and architectural choices are identified as our own position.

The Heilmeier case in brief

Graiphic Studio is being developed for a new era of industrial software engineering. As machines contribute more implementation, understanding and validating the result become strategic capabilities. This overview applies the eight questions of the Heilmeier framework; Appendix C develops the research and industrial case. [31]

1 The objective. Help engineers understand, test and correct software, including code generated by AI, through a graphical representation connected to the program that runs. The goal is to reach an accepted working system more efficiently while preserving informed human responsibility.

2 The starting point. Coding agents, textual IDEs and graphical environments already provide valuable capabilities. Engineers still spend effort reconstructing meaning across source, tools, observations and changes. Faster generation makes the quality of that complete workflow increasingly important.

3 The innovation. Use a shared graphical dataflow language to connect inspection, simulation, runtime highlighting, debugging, correction and compilation. Human authoring, agent generation and supported language imports converge on this foundation. Public FROG and the initial LLVM bridge provide a starting point; semantic fidelity and usability require continued research and validation.

4 The industrial opportunity. Acquisition, electronic instrumentation, automation and test teams could commission systems sooner, resolve defects faster and reuse specialist knowledge. The architecture is general purpose. Graiphic Studio will be a proprietary licensed solution, with a future marketplace for toolkits, hardware adapters and compiler bridges.

5 The risks. A graph may hide important effects; a translation may change meaning; observations and simulations may be incomplete. A convincing view can create false confidence. Explicit coverage, traceable builds, controlled experiments and human evaluation must address these risks, alongside adoption and maintenance costs.

6 The resources. This edition does not establish a consolidated programme budget. Investment must cover semantics and compilers, the inspection interface, runtime observation, integrations, simulation and industrial evaluation. Each stage needs a scoped and costed plan before commitment.

7 The sequence. We propose a bounded demonstrator, an industrial pilot and then broader coverage. These are evidence gates rather than announced releases. Calendar commitments depend on the selected scope, resources and validation effort.

8 The tests of success. First establish faithful mappings and actionable corrections within a defined scope. Then compare complete workflows for comprehension accuracy, defect discovery, correction time, integration effort and total cost. Acceptance thresholds should be fixed before evaluation. Gains and guarantees must be supported by results for documented configurations.

The moat we are building is the continuity of human control from generated logic to executable results.

Contents

Preface 2
The Heilmeier case in brief 3
1 From FROG to Graiphic Studio 5
2 A new era of industrial software engineering 6
3 Dataflow programming for human understanding 7
4 Three ways to create one program 10
5 Simulation and runtime debugging 12
6 An open format for agents and engineering tools 17
7 Inspecting complex software at the right level 20
8 Human in the Loop and Trustworthy AI 21
9 One solution across the instrumentation ecosystem 23
10 Compiler bridges and freedom of deployment 25
11 What LabVIEW teaches us and what changes 26
12 The philosophy and moat of Graiphic Studio 28
13 A licensed solution and a future marketplace 31
14 Results and guarantees that can be demonstrated 33
Conclusion 35
Appendix A Terminology 36
Appendix B How to read the Orville chart 37
Appendix C The Heilmeier case for Graiphic Studio 38
Appendix D References 41

1 From FROG to Graiphic Studio

An open language and a proprietary solution

The public FROG project defines a graphical dataflow programming language with an explicit program representation and a separation between language, intermediate representations, compiler bridges, runtime and development tools. Its public repository is available under the Apache 2.0 licence. These are the foundations on which Graiphic builds its product direction. [2]

The original whitepaper established a distinction that remains essential: a programming language describes a program, while an IDE helps a person work with that program. The runtime and compiler determine how it is executed on a particular target. Keeping these responsibilities clear makes it possible to improve one part of the system without making every other part dependent on a single implementation. [1]

Graiphic Studio brings these responsibilities into a coherent engineering experience. It is a proprietary solution developed by Graiphic. The open FROG language does not make Studio itself an open product, and the commercial licence for Studio does not redefine the openness of the language.

This distinction creates a useful balance. Teams can build on a documented language foundation. Graiphic can invest in the editor, inspection experience, runtime integration, compiler bridges and services that turn that foundation into a dependable professional solution.

A new purpose for the graphical interface

Graphical programming has long helped people express relationships that can be difficult to follow in a sequence of textual instructions. The new opportunity is to make those relationships available even when the initial implementation is produced by a machine.

The person’s role expands from constructing every detail to understanding intent, examining consequential choices and correcting behaviour. A good interface must support that role with more than an attractive overview. It must expose the program’s meaning, show what is known about its execution and preserve a path back to the source that can be changed.

Our innovation is the integration of these capabilities around a shared dataflow foundation. We want the engineer to move naturally from a generated proposal to a readable program, explore a scenario, investigate an unexpected value and carry a correction into the next compiled result.

This is the philosophy that guides Graiphic Studio: automation can increase the amount of software produced, while a well designed graphical language can increase the amount of that software a person can meaningfully understand and control. The product must make both gains possible within the same workflow.

2 A new era of industrial software engineering

Production becomes easier while understanding remains work

The economic argument begins with a change in the cost of producing a candidate implementation. When an agent can generate calls to a driver, a processing function or a user interface, manual entry is a smaller part of the value chain. The remaining effort shifts toward specification, integration, review, measurement and maintenance.

This is our strategic interpretation of the development of coding agents. It does not require every task to become instantaneous or every human implementation to become inferior. It requires automated production to become useful often enough that teams reorganise around it.

Programming languages remain necessary. Types, memory rules, concurrency, effects and execution semantics still determine what software does. Our thesis concerns who writes the initial implementation and how people interact with it. The language becomes a precise medium shared by tools, agents and engineers, while the interface helps the engineer retain control.

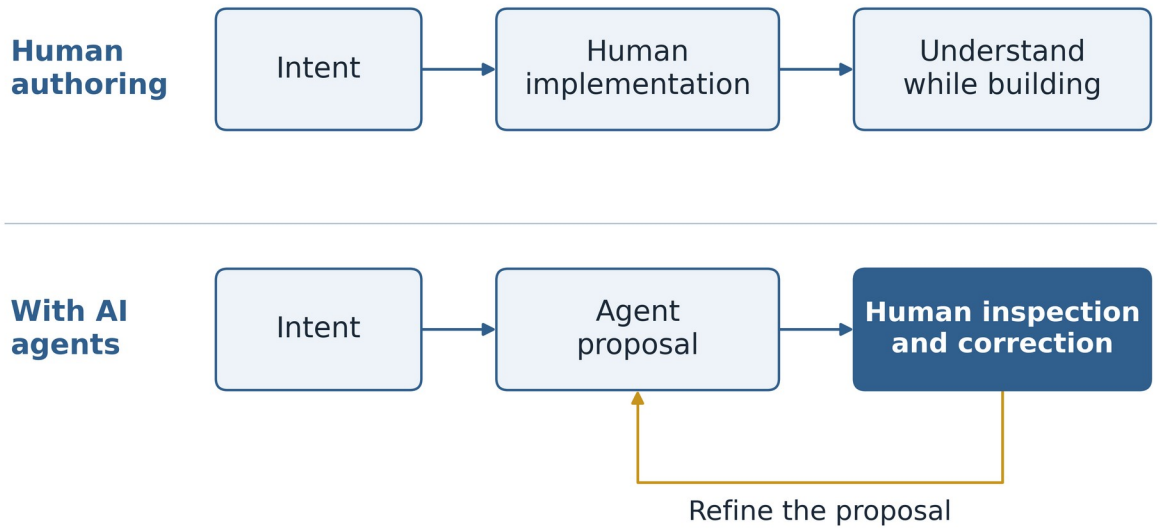


Figure 1 The value of the graphical interface changes with the production model

A conceptual shift from constructing software detail by detail to inspecting and improving generated proposals. Human expertise remains central to requirements, engineering decisions and acceptance. This figure does not assign a measured speed advantage.

The result that matters is a working system

A customer does not purchase source code as an end in itself. The customer needs an acquisition system that records the right signal, a test bench that applies the correct sequence, a service that handles information properly, or an application that performs its intended task.

Generating more code can create more integration work if the result is difficult to inspect. A productive solution therefore reduces the distance between an intention and an accepted working system. Graiphic's objective is to shorten that distance by making the generated program understandable and actionable.

The relevant economics include the time spent finding the source of an error, assessing a change, adapting an instrument interface and transferring knowledge between engineers. Savings in these activities can be valuable even when raw code generation is already inexpensive.

We expect the strongest gains where software is connected to valuable equipment, long lived applications or consequential operational decisions. In these settings, a small misunderstanding can create substantial rework. An interface that reveals the relevant behaviour can improve both productivity and confidence.

An industrial transformation with a new constraint

We believe this transition can reshape how industrial software is developed, maintained and valued. If implementation accelerates faster than comprehension and validation, those activities increasingly determine how quickly a useful system reaches operation. The ability to inspect generated software becomes part of industrial capacity: it influences how much change an engineering organisation can responsibly absorb.

This interpretation is consistent with DORA's 2025 research, which describes AI as amplifying existing organisational strengths and weaknesses and places the wider development system at the centre of returns on investment. That finding supports evaluating complete workflows; it does not establish a performance result for Graiphic Studio. [32]

For a test organisation, the opportunity may be to adapt a bench sooner or investigate a production issue with less specialist time. For an equipment integrator, it may be to reuse a verified interface across projects. For a research team, it may be to explore more experimental configurations while retaining a clear account of the software used. These are industrial outcomes that a generation speed measurement alone cannot capture.

The potential disruption also concerns the distribution of expertise. When understanding is preserved in an inspectable program, reusable scenarios and explicit component contracts, more knowledge can remain available beyond the original author. Experienced engineers can concentrate on requirements, consequential choices and acceptance while other team members investigate within a structured environment.

Our ambition is to turn this shift into a new professional solution and a durable business. The research challenge is to preserve meaning and improve human judgment at scale. The economic challenge is to deliver enough verified value to justify adoption across complete projects. These two challenges define the Graiphic programme.

3 Dataflow programming for human understanding

Dependencies become visible

The dataflow programming language is a central means of inspection in Graiphic Studio. A program is expressed through operations, typed inputs and outputs, connections and structures that give those connections meaning. The graph makes the origin of a value and the dependencies of an operation explicit.

In a dataflow model, an operation can become eligible to execute when its required inputs are available, subject to the rules of its enclosing structures and effects. Independent branches expose opportunities for concurrency. Their placement on the page does not, by itself, define execution order. NI's explanation of dataflow provides a useful account of these established principles. [3]

For the reviewer, this creates practical questions that can be answered locally. Which input determines this result? What is this operation waiting for? Which branches share a resource? Where does this value leave the system? The graphical representation gives these questions a visible structure.

A measurement example

Consider a voltage measurement. An acquisition operation provides a sample. A calibration input provides a gain. A correction operation needs both values. A limit check then needs the corrected measurement and the acceptance threshold.

With an illustrative sample of 2.4 V and a gain of 1.5, the expected corrected value is 3.6 V. This would satisfy the configured upper limit of 3.8 V. In the illustrated snapshot, execution is paused before scaling, so the downstream value and result have not yet been observed. The graph makes the dependency chain visible; runtime observations establish what has actually happened in the run under inspection.

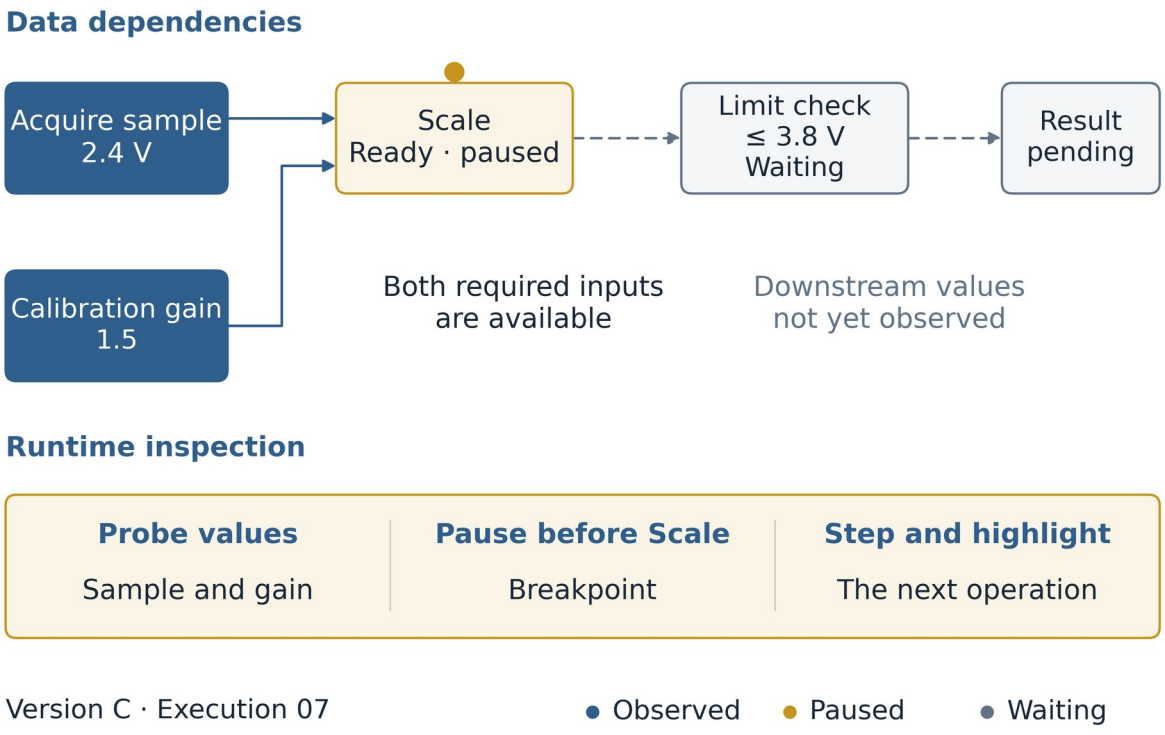


Figure 2 Dataflow connects program structure to observable behaviour

Illustrative dependencies and runtime states in a measurement workflow. Highlighting, probes and breakpoints help explain a particular execution. The diagram is a conceptual design, not a product screenshot or a timing benchmark.

If the calibration value is unavailable, the correction is waiting for a dependency. If the result is wrong, the engineer can examine the sample, gain, unit conversion and threshold without first

reconstructing their relationship from scattered source statements. This is the practical value of a dataflow interface.

Real programs include state and external effects

Useful software also contains loops, conditions, mutable state, events, errors and operations that affect the outside world. An inspection language must represent these behaviours faithfully. A clean graph that hides an important effect would undermine its purpose.

For example, configuring an acquisition task, starting it, reading samples and stopping it involve a resource lifecycle. A file write changes persistent state. A network request crosses a system boundary. The representation must retain the ordering and effect constraints that make these actions meaningful.

This is also why dataflow should not be presented as an automatic guarantee of parallel execution, deterministic timing or real time performance. The graph exposes dependencies and possible independence. The language rules, target, scheduler and external devices determine the actual execution conditions.

Hierarchy makes the representation useful at scale

An engineer should be able to inspect a subsystem as a named operation with a clear contract, then open it when more detail is needed. The same principle applies to a measurement channel, a calibration function, a document pipeline or an entire test sequence.

At large scale, a useful graphical abstraction is a hierarchy of meaningful views. It supports orientation, navigation and focused examination. The objective is to locate consequential behaviour quickly and to make detailed review efficient once the relevant area has been identified.

One language supports four engineering questions

The value of graphical dataflow extends across the work of understanding a program. Its operations, typed connections and explicit effects give the engineer stable points of reference as the task changes.

Engineering activity	Question for the engineer	Role of the dataflow representation
Inspection	What is this program designed to do?	Follow values, dependencies, conditions and external effects.
Simulation	What would happen in this scenario?	Apply controlled inputs or models at explicit interfaces and inspect the response.
Runtime debugging	Why did this execution produce this result?	Relate observed values and events to operations in the graph.
Correction and validation	Does the change now meet the requirement?	Edit the relevant logic, rebuild and compare the resulting evidence.

An engineer who follows a voltage value during inspection should be able to recognise the same dependency when a simulated input exercises it and when a runtime probe observes it. This

continuity reduces the effort spent reconstructing a different explanation in each tool. Delivering it requires explicit semantics and reliable mappings across the supported workflow.

Graphical programming therefore serves both expression and examination. Its usefulness in the AI era comes from making consequential behaviour understandable and changeable, including when the engineer did not write the initial implementation.

4 Three ways to create one program

Graphical authoring by a person

The familiar approach remains valuable. An engineer places operations, connects data, defines structures and edits the program directly. This provides a precise way to express an idea, repair a local error or explore behaviour with immediate visual feedback.

AI does not remove the need for direct manipulation. When a reviewer understands a required correction, making that correction in the same interface can be faster and clearer than describing every small change through another prompt. The product should support the engineer's judgment about which interaction is most effective.

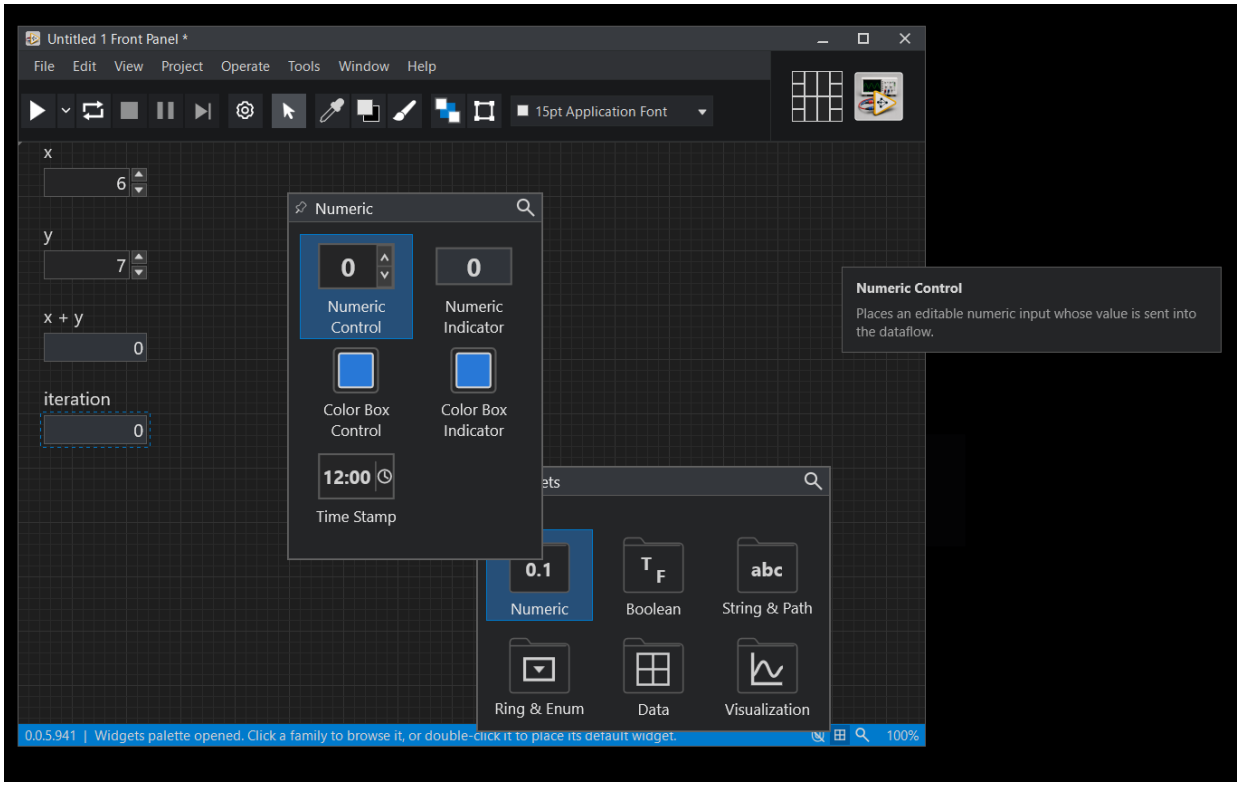


Figure 3 Building the front panel through contextual widget palettes

Numeric controls and indicators are available through the widget palette, with contextual help explaining the selected control. The front panel gives the engineer an interface for entering values and presenting results. Graiphic Studio development interface. [33]

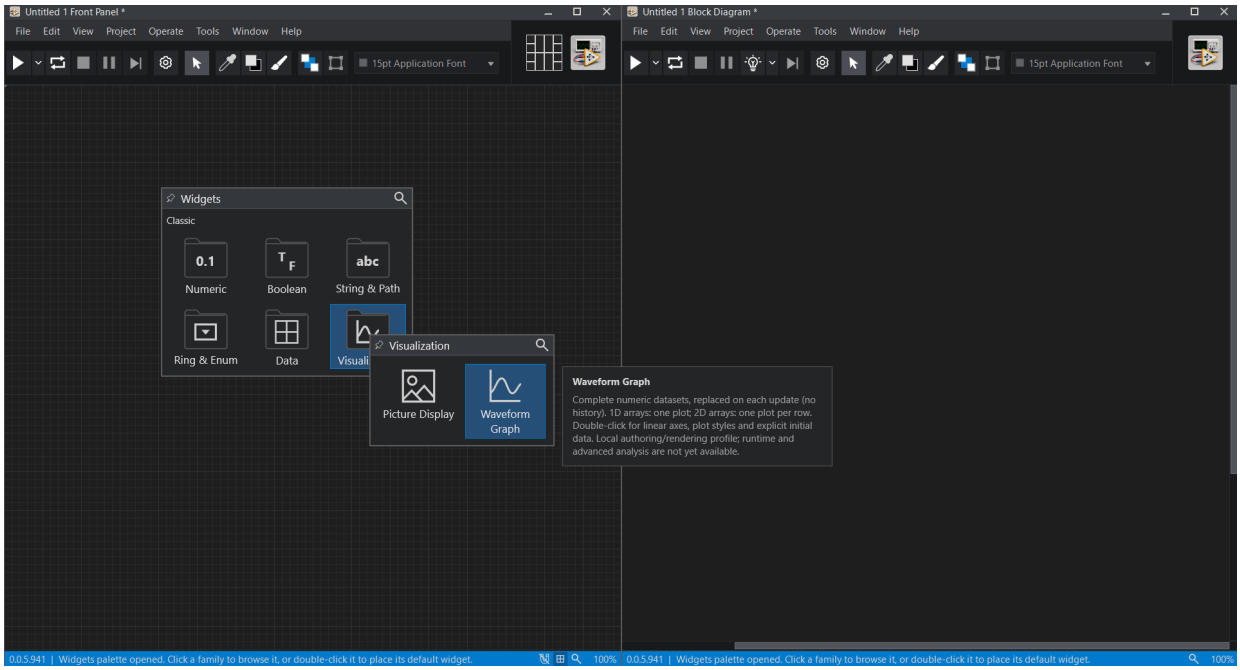


Figure 4 Discovering visualisation widgets in Graiphic Studio

The Widgets and Visualisation palettes expose Picture Display and Waveform Graph. The selected Waveform Graph is shown in its local authoring profile; its tooltip marks runtime and advanced analysis as not yet available in this development build. [33]

Generation by an AI agent

A second route is to generate the FROG program representation from an intention, specification or existing example. A text based representation can be produced and revised through normal agent tooling, then checked before it becomes an accepted program.

In this workflow, the generated graph is a proposal. Studio's role is to make its structure available for inspection, reveal relevant validation results and help the engineer improve it. The agent and the person work on a common program representation rather than on disconnected descriptions.

Imports through language bridges

A third route is to import existing software through bridges from languages such as C, C++, Rust or Java. This expands the potential of Graiphic Studio beyond code originally authored in FROG.

Each bridge must define what it can represent, which build context it needs and which transformations remain editable. Some software may support a faithful executable translation. Other software may initially support a navigable inspection view, with external operations retained behind explicit interfaces. Unsupported constructs must remain identifiable.

Knowing the definitions of a language's functions is insufficient to guarantee universal conversion. Memory aliasing, exceptions, concurrency, macros, undefined behaviour, dynamic loading and external libraries can affect meaning. A credible bridge preserves the supported semantics and states the boundaries of its coverage.

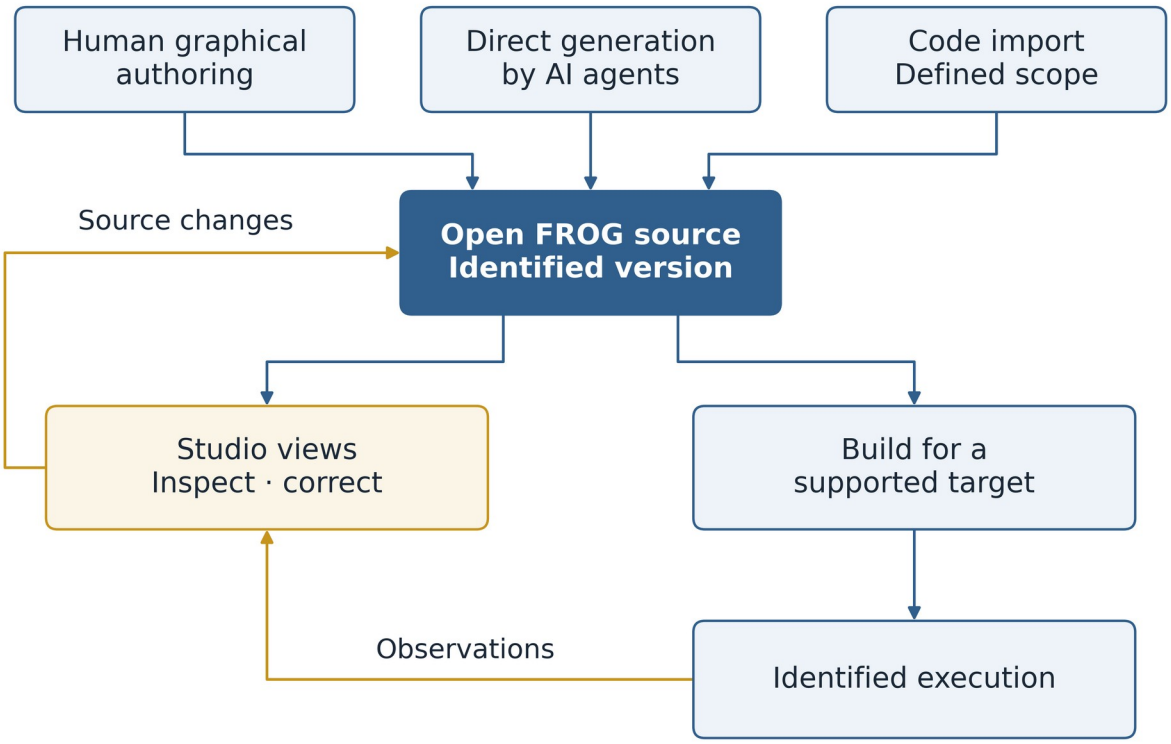


Figure 5 Three authoring routes converge on an inspectable program

Human authoring, agent generation and scoped imports feed a shared FROG representation. Inspection, correction, compilation and runtime observations form a continuing engineering workflow. Import coverage depends on the bridge and source environment.

Editing must preserve a clear source of truth

For each workflow, the engineer needs to know which artefact is authoritative. If imported C++ remains the source of truth, the system must define how a graphical edit is reflected in that source. If an import becomes a FROG program, that transition should be explicit.

Our direction is to make supported edits operational. A correction should have a defined effect on the accepted source and the next build. This requirement separates a useful development abstraction from a diagram that merely documents an earlier version of the software.

5 Simulation and runtime debugging

Experimentation and observation share the graph

Inspection reveals the structure of a program. Simulation lets the engineer choose conditions and investigate the resulting behaviour. Runtime debugging helps explain a particular run, whether its environment is simulated, connected to real equipment or deliberately combines both.

Our product direction brings these activities together around the graphical dataflow language. The engineer should be able to inspect a dependency, exercise it with a controlled scenario, observe the result and correct its implementation. The simulation capabilities described here define a development direction; their supported scope will be stated as they become available.

Explore behaviour before connecting the bench

Consider a test sequence that configures a power supply, acquires a voltage, applies calibration, evaluates a limit and records a result. Before occupying the complete bench, the engineer should be able to ask what happens when the sample is outside the expected range, the acquisition is late or an instrument does not respond.

The graph makes the relevant interfaces visible. A scenario can supply known samples to a processing function or replace a supported device interaction with a simulated adapter. The rest of the program can then be examined under stated conditions. The intended gain is earlier feedback on logic and error handling, with less dependence on the availability of every physical component.

Three levels of capability need distinct contracts.

Simulation level	What it makes possible	What the configuration must define
Controlled inputs	Exercise logic with synthetic samples, recorded input sequences and boundary values.	Data types, units, initial state, input order and relevant timing.
Simulated interfaces	Exercise acquisition, instrument or service interactions through substitute adapters.	Responses, state changes, delays, errors and the extent of the simulated interface.
Physical system models	Explore a controller or test sequence against a model of a sensor, motor, power supply or process.	Model assumptions, parameters, time semantics and numerical solver where required.

A dataflow graph provides a structure for connecting these elements. Physical fidelity comes from the chosen models and their validation. A simulated instrument can reproduce a documented timeout response without reproducing the analogue behaviour, thermal limits or timing accuracy of the real device.

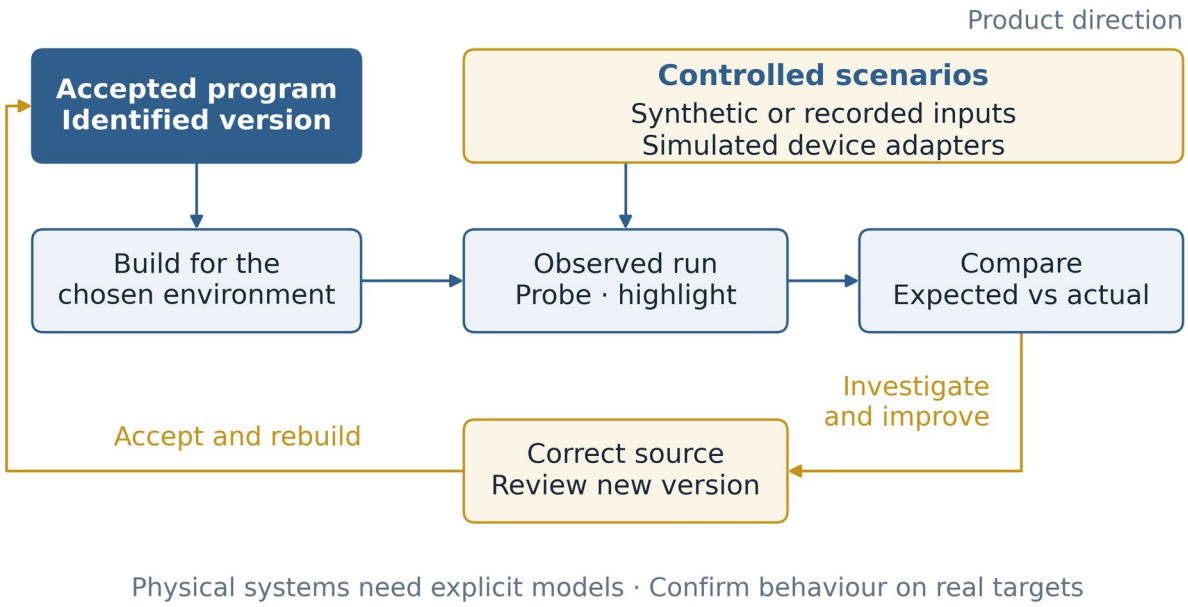


Figure 6 A controlled scenario connects experimentation to correction

The intended workflow applies defined inputs or simulated interfaces to an identified program and build, compares observed behaviour with expectations, then carries a correction into a new build. Physical models and real target validation have separate scopes. Simulation capabilities are part of the product direction.

A scenario makes an engineering question testable

For the illustrative measurement in Chapter 3, a known sample and gain establish an expected result. Additional scenarios ask whether the sequence handles a failed measurement and an unavailable resource correctly. Expected outcomes come from the application requirements and should be defined before the run.

Scenario	Question to examine
Sample of 2.4 V with gain 1.5	Does the program produce 3.6 V and apply the configured 3.8 V upper limit correctly?
Sample outside the accepted range	Does the limit decision use the corrected value and report the intended outcome?
Delayed acquisition	Does the sequence wait or time out according to its requirement, without using an older sample as a new one?
Instrument does not respond	Does the error reach the intended handling path, with resource cleanup where required?
Result storage fails	Does the program report that the measurement result was not successfully recorded?

These scenarios turn review into an experiment. The engineer can inspect why a condition exists, observe which path it takes and compare the result with the requirement. Once a defect is corrected, the scenario becomes a reusable regression check for subsequent changes.

Keep experiments connected to evidence

Each experiment should identify the program version, build configuration, initial state, scenario, adapter or model versions and expected outcomes. The interface should distinguish simulated values from physical measurements and identify any effects still connected to real equipment.

Replaying recorded observations helps explain an earlier run. Feeding recorded inputs into a new run tests the current program under a chosen input history. These are different activities. Repeatable execution can also require control over clocks, random values, external events and concurrency; a data file alone does not establish deterministic replay.

The same source logic may be exercised through different builds and targets. Observations from a host build do not automatically establish timing or behaviour on the deployment target. Established SIL and PIL workflows illustrate this distinction: MathWorks documents execution of generated code on a development computer for SIL, and on a target processor or instruction set simulator for PIL. [30]

For Graiphic, the practical objective is a clear progression from an inspected program, through controlled experiments, to validation on the supported target and equipment. The expected savings come from discovering more problems before full bench integration and carrying useful scenarios into later verification.

Runtime highlighting makes behaviour tangible

A graph explains possible behaviour. Runtime observations explain what happened in a particular execution. Graiphic Studio’s product direction joins these two forms of understanding through execution highlighting and focused debugging tools.

The engineer should be able to see which operation is active, which branch was taken, which input is missing and which value reached an output. These observations make a dataflow program easier to understand and provide a practical path into debugging.

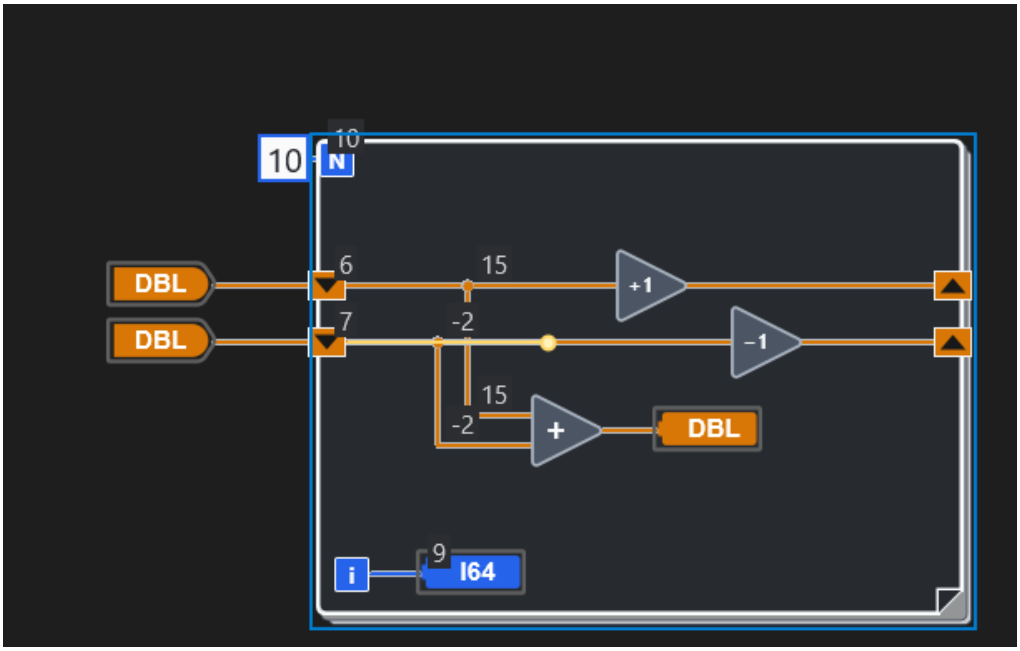


Figure 7 Values become visible within the dataflow graph

The loop view includes numeric value annotations, a highlighted wire marker and an iteration value. Execution information is displayed beside the operations and connections under inspection. This screenshot records one view of a Graiphic Studio development build. [33]

Highlighting shows observations from a run; simulation supplies a controlled environment for that run. Their combination lets an engineer follow a scenario through the dataflow graph and understand why an expected path was or was not taken.

Visual debugging has a proven precedent. LabVIEW documents execution highlighting, probes, breakpoints and single stepping, and explicitly notes that highlighting slows execution. We recognise this inheritance. Graiphic's opportunity is to integrate such understanding with open program representations, AI authoring and a broader compilation architecture. [4]

Probes and breakpoints support focused investigation

A probe answers a specific question about a value at an observation point. For a signal, this may involve a sample, an array, a timestamp or a selected interval. For a structured record, it may involve a field whose value determines a decision. The interface should make the relevant context visible without forcing the engineer to expose every value at once.

A breakpoint creates an opportunity to investigate a state before execution proceeds. Conditional breakpoints can focus attention on an unexpected value or error. Stepping into an operation, over a known component or out of a subsystem allows the engineer to choose the depth of examination.

These tools become more useful when they are linked to the same graph used for review and correction. The engineer can follow a dependency, observe its value, identify an incorrect assumption and edit the relevant operation without changing mental models at every step.

Observations belong to a version and a run

A trustworthy interface must distinguish a current observation from retained data, an unavailable value and an operation that has not been observed. A missing measurement is not zero. A value collected before an edit does not automatically describe the edited program.

Our design principle is to associate observations with the source version, compiled artefact, execution session and relevant event or iteration. This makes it possible to explain the relationship between the program on screen and the evidence beside it.

Compiler optimisation may merge, remove or reorder operations while preserving the supported program semantics. Consequently, the relationship between source elements and runtime events may be more complex than one visible node per machine instruction. The interface should communicate the available mapping and observation coverage clearly.

Probes, stepping and node level highlighting depend on the compiler bridge and execution environment. Their availability and granularity must be stated for each supported configuration, including optimised builds.

Debugging and performance answer different questions

Interactive highlighting and pausing help explain behaviour. Performance analysis requires measurements made under stated conditions, including observation overhead. A slow animation cannot establish the timing of the deployed system.

For a test bench, pausing application logic also does not imply that physical equipment has stopped. Instrument state, independent protection and the behaviour of control systems remain part of the engineering context. The debugging interface should help the engineer understand those boundaries.

The intended result is a shorter and more reliable learning cycle: observe the unexpected behaviour, locate its cause, make a correction, rebuild and compare the next execution. This is a concrete productivity gain that can be measured on representative tasks.

6 An open format for agents and engineering tools

JSON supports an accessible integration path

FROG’s textual expression uses JSON. This is a practical foundation for agents and tools because it can be generated, parsed, validated, compared and stored using widely available techniques. The language specification gives that representation a meaning beyond the JSON syntax. [1]

A structured generation interface can constrain the form of an output. OpenAI’s Structured Outputs documentation provides one example of generating content that adheres to a supported JSON schema. Such a facility helps establish structural validity; it does not establish that the generated algorithm fulfils its intended purpose. [5]

Our engineering objective is therefore a sequence of checks: valid representation, valid language structure, meaningful types and connections, supported execution semantics, and evidence that the

resulting behaviour meets the requirement. These checks make generation useful in a professional workflow.

An open representation for direct agent integration

LabVIEW VIs use a binary format, as NI's software configuration management documentation explains. NI also provides VI Scripting capabilities. An agent may interact through these APIs or through an intermediate representation rather than generate a VI's binary bytes directly. [6][7]

The advantage we seek with FROG is a direct, documented program representation that can participate naturally in text based tooling and validation. We should evaluate that advantage through reproducible authoring and review tasks. A file extension alone cannot establish the relative quality of an entire development workflow.

The graphical layout and the executable meaning also deserve distinct treatment. Moving an operation on the canvas should not obscure a semantic change, and an important semantic change should remain visible even when the layout looks similar. This supports more useful comparisons between program versions.

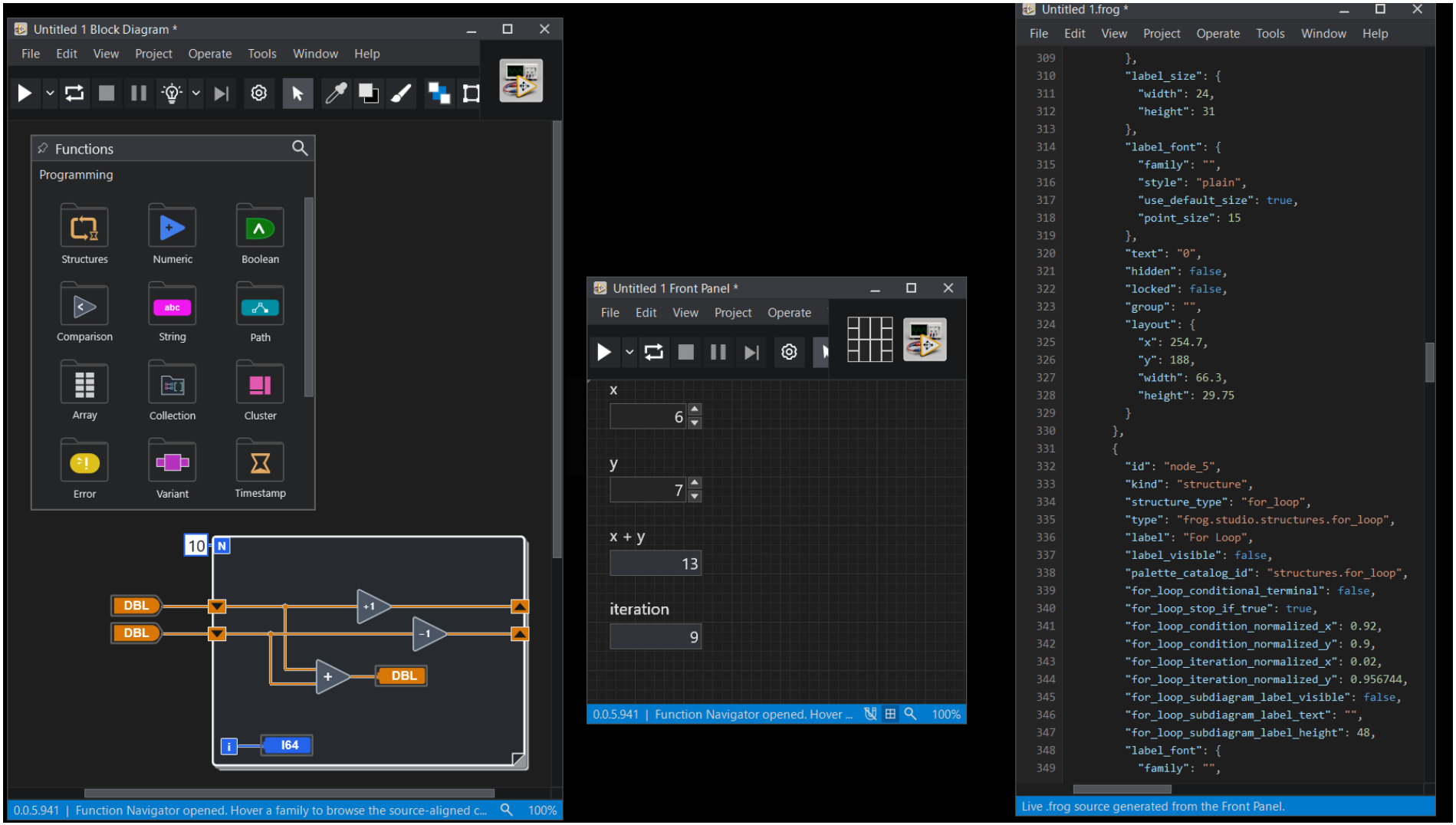


Figure 8 Graphical logic front panel and FROG source in one workspace

A loop diagram, numeric controls and indicators, and the textual .frog source are visible together. The screenshot illustrates the relationship between graphical authoring and the program representation. Graiphic Studio development interface supplied by Graiphic. [33]

7 Inspecting complex software at the right level

The first glance should reveal where to look

The ambition to make very large programs inspectable begins with selecting the right level of abstraction. A single diagram containing millions of operations would overwhelm the reader. A well structured overview can instead reveal major subsystems, external dependencies and the paths that connect important inputs to consequential outputs.

The engineer can then descend into the relevant area. An acquisition overview may lead to a channel configuration, a calibration function and the exact conversion used in a decision. A service overview may lead to a data transformation, an access check and a network destination.

This is the meaning of rapid graphical inspection in our product direction: faster orientation followed by focused, evidence based examination. Human attention is directed toward the relationships that matter for the question being asked.

Effects and changes deserve dedicated views

Some of the most important review questions concern effects outside a local computation. Does this change add a network request? Does it alter a file path, a device command or a permission boundary? Does it change the order in which a resource is opened, used and released?

Consider a document processing application. A new version still reads documents and produces a summary, but it also introduces a network export. At a high level, the main processing path may look familiar. An effects view should make the new destination explicit and allow the reviewer to inspect its condition, payload and purpose.

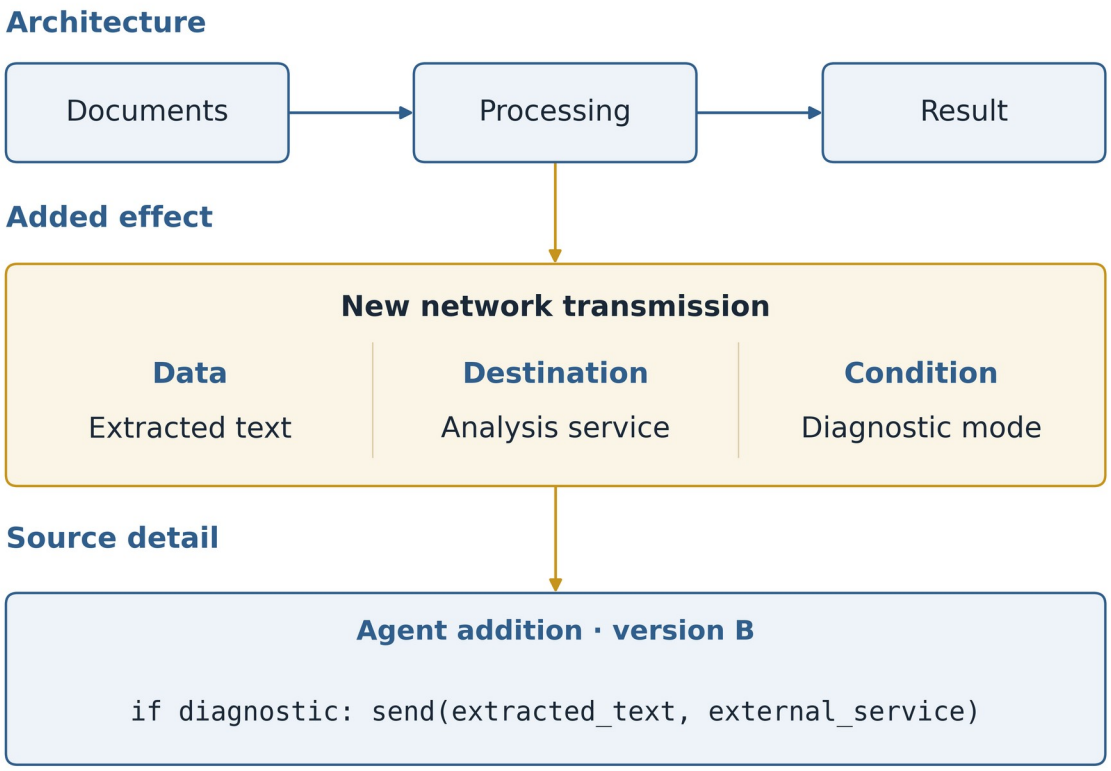


Figure 9 A focused view makes a consequential change easier to examine

An illustrative document application gains a network effect. The inspection task is to locate that effect, understand its conditions and follow it to the relevant source. This general purpose example complements the instrumentation workflows.

Security and optimisation require evidence

A graphical interface can help reveal a suspicious dependency or unnecessary operation. It cannot guarantee the absence of a backdoor or prove every optimisation correct merely by displaying a simplified view.

Our approach is to connect graphical inspection with analysis, tests and observations. A reviewer who notices an unexpected network path should be able to examine the implementation and the conditions under which it executes. A reviewer who suspects a bottleneck should be able to connect the relevant operations to measured duration, resource use or blocking behaviour.

This creates a useful relationship between human understanding and automated checks. Analysis can identify areas that deserve attention. The graph can make those areas understandable. The engineer can decide on the correction, then verify the result through the appropriate evidence.

8 Human in the Loop and Trustworthy AI

Human participation must include understanding and action

Human in the Loop is meaningful when a person can understand the relevant proposal, examine its consequences and change the outcome. An approval button placed after an opaque generation process provides little support for informed responsibility.

Graiphic Studio is being developed to make participation operational. The engineer should be able to inspect a proposal, request or make a correction, compare versions and accept a specific result. The interface should preserve the connection between that decision and the program that is subsequently built.

Trustworthy AI, in this context, is an engineering objective. It concerns how AI contributions enter the software lifecycle, how their consequences become visible and how evidence supports human decisions. It is not a label that removes the need to establish correctness for the application.

Acceptance applies to a particular version

Suppose version A has been executed and observed. An agent proposes version B. The engineer identifies an incorrect threshold and produces version C. Acceptance concerns C. Observations from A can inform the review, but they do not establish the behaviour of C until the relevant build and execution evidence exists.

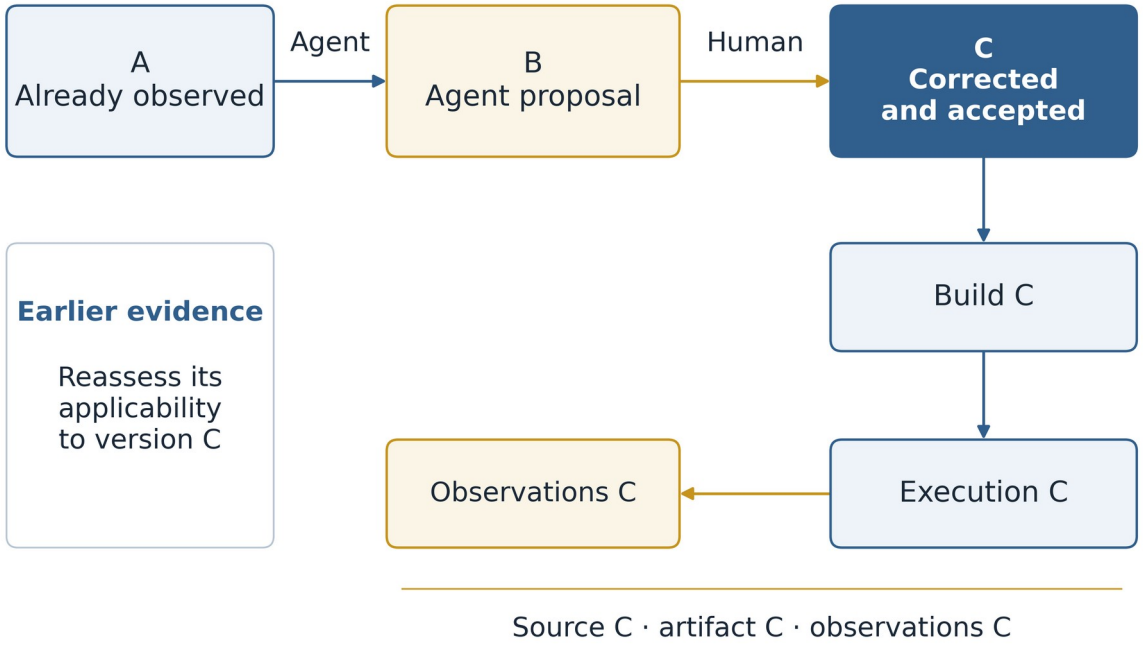


Figure 10 Human acceptance is connected to the version that is built and observed

A previously observed program, an AI proposal and a human correction are distinct states. The intended workflow links the accepted source to its build and subsequent runtime evidence.

This principle has consequences for the product. Version identity, dependencies, compiler configuration and relevant execution context belong in the chain of evidence. An interface that preserves these relationships helps a team explain what was reviewed, what changed and what was actually tested.

Responsibility remains connected to expertise

The graphical abstraction should make expertise more effective. It can help a measurement engineer examine units, ranges and calibration; a software engineer inspect state and resource handling; and a reviewer understand the evidence supporting acceptance.

For strategic applications, the appropriate review process depends on the system and its requirements. Graiphic's role is to provide an inspectable engineering foundation that can support those processes. Product specific assurances and any qualification claims must be established for their stated scope.

The commercial value is stronger when responsibility is supported by a usable workflow. Teams gain a clearer way to investigate changes, share understanding and preserve the reasoning behind an accepted result.

9 One solution across the instrumentation ecosystem

A general purpose architecture with a concrete industrial application

Graiphic Studio is general purpose. Its ambition includes software that handles documents, services, data processing and other application domains. Electronic instrumentation is a particularly powerful demonstration because software immediately connects to measurable behaviour in the physical world.

A modern bench may contain an oscilloscope, a power supply, a signal generator, switching, acquisition channels, sensors and an industrial controller. The engineer needs these components to work together as one system. The choice of a programming environment should follow that objective.

The following examples identify documented integration routes in the instrumentation ecosystem. They describe possible foundations for adapters; they are not a catalogue of already qualified Graiphic Studio integrations.

Ecosystem	Documented integration route	Example role in a system
NI	NI-DAQmx C API and Python package. [11]	Acquisition tasks, channels, timing and sample transfer.
ADLINK	DAQ software libraries and DASK family interfaces. [12]	Acquisition and device control.
UEI	UEIDAQ Framework and device programming APIs. [13]	Distributed or modular acquisition and control.
Gantner Instruments	GInsData library and documented interfaces. [14]	Access to measurement systems and recorded data.
Beckhoff	TwinCAT ADS interfaces, including TcAdsDll for external applications. [15]	Communication with PLC and control system variables.
Keysight	VISA I/O, SCPI commands and instrument drivers, including IVI options. [17][18]	Power supplies, oscilloscopes, analysers and other instruments.
Rohde & Schwarz	SCPI and instrument driver examples across several programming environments. [19]	Remote control, measurement synchronisation and result retrieval.

Native interfaces and protocols provide the connection

Native library interoperability is part of the FROG architecture. It provides a route to functions exposed by hardware software libraries. The practical contract includes the calling convention, types, ownership, supported operating system, architecture and error behaviour. [1]

A DLL is one form of native integration. Instruments may also expose a communication protocol or a service. VISA provides an I/O interface; SCPI describes commands used by many instruments. Drivers can offer a higher level abstraction, with coverage that depends on the instrument and driver. Keysight documents these distinctions and the limits of interchangeability. [17][18]

The same precision matters for Beckhoff. An external application communicating through ADS is different from code executing within a TwinCAT real time module. Beckhoff documents constraints on using third party libraries in that environment. A Graiphic bridge must preserve the distinction between controlling a target and executing within it. [16]

A mixed bench illustrates the intended result

Consider an illustrative system with a Keysight power supply, NI acquisition and Beckhoff I/O. The program establishes the device configuration, checks the necessary operating conditions, applies a stimulus, acquires a signal, evaluates a limit and records the result.

A dataflow view can expose which values and conditions permit each stage to proceed. Runtime observations can show the measured value, the selected range, the active stage and an instrument error. A focused view can reveal whether the final result used the intended calibration and threshold.

Supported simulated adapters would let engineers exercise selected parts of this sequence before every instrument is available. The scenario and interface contract should remain visible when a real device replaces its simulated counterpart, so the team can identify which assumptions still need confirmation on the bench.

An adapter for ADLINK, UEI or Gantner could provide an alternative acquisition integration where its supported capabilities match the application. Instrument interchangeability still depends on measurement characteristics, timing, triggers and software behaviour. The product opportunity is to make those interfaces explicit and reusable.

The expected gains are reduced integration effort, faster diagnosis and better reuse between benches. They should be demonstrated on real configurations with the device models, driver versions and operating conditions recorded.

Agents can also connect directly to instruments

MCP is relevant beyond IDE integration. The public Rohde & Schwarz RsInstrument project documents an MCP server and tools for instrument discovery and SCPI operations. This provides a concrete example of connecting agent workflows to instrumentation services. [29]

For Graiphic, such connectivity complements the inspection architecture. The engineering question remains which actions are permitted, how they fit the program’s dependencies and what observations establish their result. The interface should make those relationships understandable across the complete system.

10 Compiler bridges and freedom of deployment

LLVM is the first proof of feasibility

Graiphic's compiler direction begins with LLVM as a first bridge and proof of feasibility. It establishes an initial route from the language's meaning toward executable software. Our ambition extends to further compiler bridges and supported execution environments.

The architectural objective is to retain a stable relationship between the program being inspected and the compilation route used to execute it. FROG's meaning should remain distinct from the implementation choices of a particular backend. This gives us a foundation for improving compiler support without redefining the entire user experience.

LLVM itself exposes target specific considerations such as data layout in its intermediate representation. MLIR's architecture illustrates another established approach to representing and lowering computations through intermediate forms. These references support the importance of explicit representation and transformation contracts; they are not evidence that every target is already supported by Graiphic. [20][22]

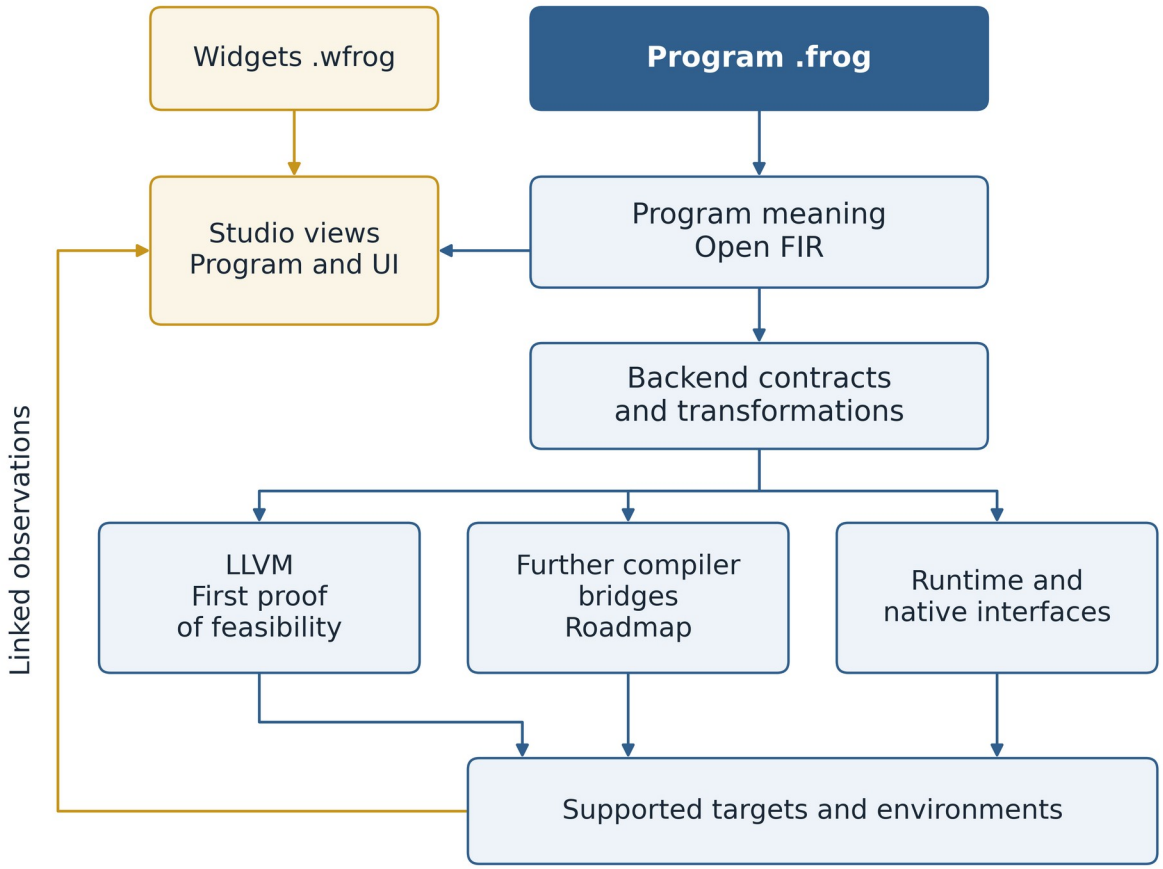


Figure 11 Compiler bridges connect a shared program meaning to supported targets

LLVM is the first proof of feasibility. Further bridges form part of the roadmap. Each route must define its supported semantics, target conditions and observation capabilities, with useful information returned to Studio.

A bridge needs a clear contract

A compiler bridge must state which language features it accepts, how it represents external calls, which runtime services it needs and which target configurations it supports. It should also define the information returned for diagnostics and inspection.

For future targets, this may involve a vendor compiler, an embedded toolchain or a specialised execution environment. Each route creates an engineering task: preserve the relevant semantics, integrate the platform and establish evidence for the claimed result.

Deployment freedom therefore grows through qualified bridges. Once a target and its requirements are supported, the product can make the build and deployment workflow more direct. The ambition is rapid delivery across a growing range of systems, with clear compatibility boundaries.

Importing existing code requires its build context

A language bridge needs more than an isolated source file. Include paths, macros, language options, dependencies and target assumptions can change what that file means. Clang's compilation database documents how tooling can receive the command context used to compile each translation unit. [21]

For a C++ import, preserving meaning may require information about object lifetimes, exceptions and aliasing. Rust introduces ownership and lifetime semantics that an import must respect. Java may rely on a virtual machine, reflection and dynamic behaviour. These differences explain why bridge development must proceed through explicit capabilities.

The long term opportunity remains substantial. A common inspection experience could serve programs originating from several languages while supported compilers and runtime integrations provide multiple execution routes. The quality of that connection will determine the value of the abstraction.

11 What LabVIEW teaches us and what changes

A valuable inheritance

LabVIEW demonstrated the usefulness of a graphical dataflow language for engineers working with acquisition, measurement and control. Its contribution includes visible dependencies and practical visual debugging. These ideas remain relevant to Graiphic's direction. [3][4]

The strategic change concerns the reason to adopt a graphical environment. If an engineer can ask an agent to create an acquisition program through a documented API, ease of manual graphical construction becomes less decisive for that task. The interface needs to justify its value across the wider engineering lifecycle.

We therefore retain the dataflow insight and develop its role in inspection. The graph helps people understand software regardless of whether its first version was created by a person or an agent.

MCP and AI assistance are real responses

The incumbent ecosystem is responding. NI documents prompt based VI creation in LabVIEW 2026 Q3 through Nigel, initially focused on measurement code for connected hardware. Nigel can also connect as a client to MCP servers and use their tools. This is distinct from an external agent controlling the LabVIEW IDE. [8][27]

Community and company projects provide additional routes. Jan Göbel's G-AI describes project and diagram inspection, together with incomplete VI creation capabilities, using Claude Desktop as a tested client. Zühlke's independent labview-mcp describes reading, generating, editing and running VIs through tools that include AIXML and a LabVIEW service. Its installation guide explicitly covers Claude and Codex. The project identifies research status and reliance on an unpublished NI interface. [9][10][28]

These developments show that a binary VI format does not prevent AI integration. They also create useful opportunities to maintain existing applications, access established instrument logic or automate parts of a LabVIEW workflow.

Our judgment concerns the purpose of the system

Our strategic view is that asking Claude or Codex to generate LabVIEW does not, by itself, create a new reason to choose LabVIEW. The objective remains controlling the bench, acquisition cards and instruments, then obtaining a reliable result.

For a new application, an agent can work with available APIs and conventional languages as well as graphical tooling. NI's Python package, for example, provides access to NI-DAQmx through the driver's C API. NI hardware and LabVIEW are therefore distinct choices in that context. [11]

When a project must remain in LabVIEW, an experienced LabVIEW engineer is an established route to maintaining the architecture and delivering changes. Agent assistance may complement that work. We expect agent to LabVIEW workflows to be primarily incremental around such requirements unless they demonstrate a compelling advantage for new projects.

The commercial question is whether the chosen environment delivers the most useful result for the total engineering effort required. Our thesis is that this comparison will increasingly favour solutions designed around AI generation, human inspection and freedom of integration.

Why we expect the traditional position to erode

Our forecast is that a graphical environment whose main economic advantage is making manual programming accessible will face increasing pressure. If an agent can produce an acceptable implementation through another interface, customers will compare total engineering effort, licence cost, portability and long term maintenance.

Under that scenario, LabVIEW could progressively lose its role as the default entry point for new instrumentation software. Continued dependence on an installed base could lead to contraction and, eventually, disappearance from parts of the new project market. This is the sense in which we anticipate the traditional position disappearing.

It is a conditional forecast, not an announced end of the product. Existing systems, specialised capabilities, support and a successful response to AI can sustain adoption. Graiphic's strategy must succeed through its own results rather than depend on a competitor's decline.

12 The philosophy and moat of Graiphic Studio

A distinctive philosophy of graphical programming

The moat Graiphic is building is the continuity of human control from generated logic to executable results. Our philosophy gives the graphical dataflow language a central role in making AI generated software understandable, testable and changeable by the people responsible for it.

The engineer should be able to follow a requirement into the relevant logic, explore its behaviour through a scenario, inspect an observed result and carry a correction into the next build. The graph provides a shared vocabulary for these activities. Human in the Loop becomes a productive engineering practice because understanding can lead directly to action.

A readable graph gains value when its connections have precise meaning. A simulation gains value when it exercises the logic under review. Runtime evidence gains value when it identifies the version, operation and conditions involved. A graphical edit gains value when its effect survives the path to compilation. Graiphic's innovation is to develop these relationships as one coherent solution.

This philosophy applies across the three authoring routes. A person can draw a program, an agent can generate FROG, or a supported bridge can bring existing software into the inspection workflow. In each case, the engineer needs the same ability to understand consequential behaviour and act on that understanding.

A research programme with testable questions

Our ambition depends on technical and scientific questions that deserve explicit investigation. An appealing interface becomes valuable when it preserves consequential behaviour and helps people make better decisions. The following questions connect product development to a research programme.

Preserving meaning across representations. How can supported imports, graphical edits and compiler transformations retain the relevant types, state, aliasing, ordering and external effects? Progress requires defined coverage and comparisons against reference behaviour, including difficult cases. A diagram independently generated by an AI cannot establish the meaning of the executable without a validated connection to its build inputs.

Making complexity understandable. Which hierarchies and views let an engineer find important behaviour without hiding it? We need to evaluate accurate explanations, detection of consequential changes and confidence in those judgments. Larger graphs and more visible nodes do not automatically improve comprehension.

Connecting the graph to optimised execution. Which observations remain meaningful after transformations, concurrency and target specific scheduling? Runtime highlighting and probes must

identify what was observed, where the mapping is incomplete and how observation affects execution. Their usefulness should be measured alongside their overhead.

Transferring evidence from simulation to operation. Which claims can be supported by controlled inputs, simulated interfaces and physical models, and which still require target or hardware validation? The research task includes making that boundary understandable so that a successful scenario does not invite an unsupported conclusion about every operating condition.

Improving human control. Does the integrated workflow help engineers recognise a fault, choose an appropriate correction and verify it more effectively than the defined alternatives? Evaluations should examine both speed and correctness, including overconfidence. Human in the Loop becomes an engineering result when informed intervention measurably improves the accepted system.

These questions give the programme a clear route from research to industrial application. Demonstrated answers can become reusable mappings, interaction techniques, scenarios and validation methods. Their accumulation is part of the technological moat.

The moat accumulates through engineering depth

The first source of defensibility is **semantic fidelity**. The representation must preserve relevant types, dependencies, state, control structures and external effects. Mappings through imports, intermediate representations and compiler bridges must remain reliable as supported languages and targets evolve. The accumulated implementation and validation work becomes a substantial technical asset.

The second is **usable human control**. Hierarchical views, meaningful change comparisons, scenario configuration, runtime highlighting and focused probes must work together without overwhelming the reviewer. The product should reveal what is known, what has changed and where a person can intervene. Achieving this consistently across complex programs requires sustained product and engineering work.

The third is **actionable correction**. The accepted program, graphical edits, compiler inputs and observations need a clear relationship. Preserving that relationship across real workflows makes the abstraction useful for maintaining software as well as explaining it. Engineers can build knowledge that remains connected to the software they change.

The fourth is **reusable integration**. A qualified instrument adapter, compiler bridge or domain toolkit extends the range of systems that benefit from the same experience. Scenarios, regression cases and documented limitations can improve those components over time. Specialists can contribute capabilities that would otherwise be expensive for each customer to rebuild.

These assets reinforce one another. Better mappings improve inspection. Better inspection helps identify integration defects. Reusable scenarios preserve what has been learned. A growing ecosystem brings more useful work into the same environment. This is how a distinctive philosophy can become a durable technological and commercial advantage.

An open language supports an exclusive product experience

JSON and graphical dataflow are foundations that others can use. Graiphic's defensibility will come from the depth, reliability and usability of the integrated solution built on them. We intend to develop exclusive Studio capabilities while keeping the FROG language foundation open.

For customers, the value is continuity: less effort reconstructing meaning between generation, inspection, experimentation, debugging and deployment. For Graiphic, the corresponding assets are product expertise, validated mappings, maintained integrations and the ecosystem built around them. The commercial licence funds that work; the marketplace can expand its reach.

An Orville positioning chart for the new context

The adapted chart uses two dimensions relevant to the new context: the integration of AI into software production, and the continuity of human control from dataflow to runtime. It compares approaches qualitatively and shows the architecture Graiphic intends to establish.

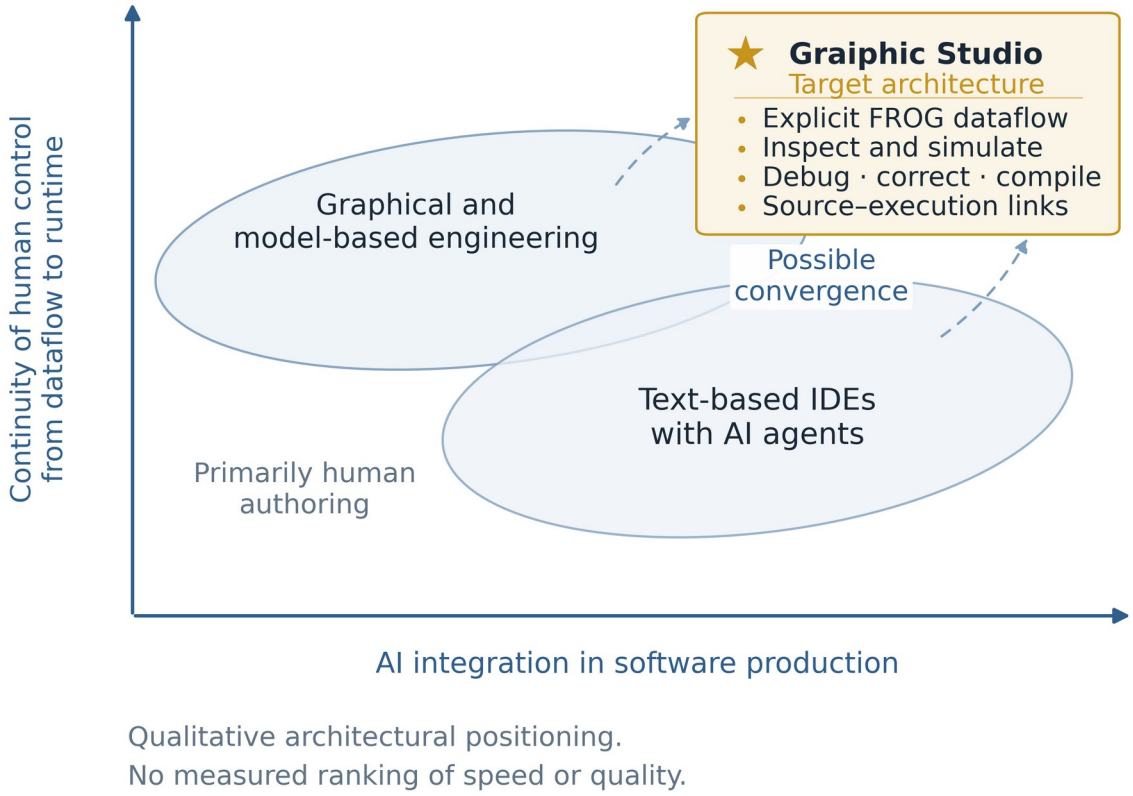


Figure 12 An Orville style map for the AI software era

A qualitative positioning framework built around AI integration and continuity of human control. Graiphic Studio marks a target architecture connecting dataflow inspection, simulation, debugging and correction to compilation. The areas overlap because tool families can combine capabilities. See Appendix B for scope and interpretation.

Text based IDEs with agents and graphical engineering environments already contribute relevant capabilities. MathWorks documents model to generated code tracing and code inspection, and presents AI assistance for Simulink. Anthropic also offers AI code review. These examples establish that adjacent approaches are active and capable. [23][24][25][26]

Our differentiation must therefore be demonstrated through complete workflows. The decisive questions are how accurately an engineer understands the generated program, how efficiently scenarios expose an incorrect assumption, and how quickly a verified correction reaches the supported target. These results will establish the strength of the Graiphic moat.

13 A licensed solution and a future marketplace

The commercial product

Graiphic Studio will be distributed under a commercial licence. The intended proposition is an integrated professional solution for creating, inspecting, simulating, debugging and deploying software with the FROG foundation.

The licence supports continued investment in the product experience, compiler and runtime integration, maintenance, documentation and customer support. Commercial terms will define the available editions, supported configurations and service commitments. This whitepaper does not prescribe a price or a particular subscription or perpetual licence structure.

The commercial promise is to reduce the total effort required to reach an understood, accepted and deployable result. Customers pay for the value of the complete engineering experience: faster investigation, reusable integrations, dependable maintenance and support. We will demonstrate that value through representative projects and measurable engineering savings.

From customer value to durable revenue

The economic potential extends across the lifecycle of industrial software. A customer may benefit from shorter commissioning, faster diagnosis, less repeated integration work or easier maintenance of an existing system. An instrumentation toolkit can create recurring value when the same supported capability is reused across benches, products or sites.

That value should be estimated from the work actually improved. An evaluation can record engineering effort avoided, external expenditure reduced and documented benefits from earlier delivery or less rework. It must also include licence fees, integration, training, migration, ongoing maintenance and any additional infrastructure. Benefits should be counted once, with the assumptions and comparison period made explicit.

Released engineering capacity and cash savings are different outcomes. A team that completes an investigation sooner may use the time to run another experiment or improve test coverage. That is a productive gain even if payroll expenditure is unchanged. A financial claim should distinguish this capacity from expenses that are actually avoided.

Graiphic’s revenue model captures part of the value through the Studio licence and, as the ecosystem develops, distribution and support of reusable components under defined commercial terms. The strongest basis for durable revenue is repeated customer benefit on supported tasks. The marketplace can extend that benefit by allowing specialists to serve needs beyond Graiphic’s own integration capacity.

This is the economic hypothesis to test through industrial pilots. We do not infer a market size or a universal productivity multiplier from the existence of coding agents. We aim to establish a

repeatable relationship between technical innovation, measurable engineering gains and a commercial proposition customers choose to sustain.

An open foundation supports independent participation

FROG remains the open language foundation. Third parties should be able to develop useful tools and integrations around its documented representation, subject to the applicable licences.

Graiphic Studio can offer an integrated proprietary experience on that foundation. This creates room for product specific innovation and services without requiring the language itself to become closed. It also gives customers a clear distinction between the program format, the editor licence and the terms of an optional third party component.

A marketplace for reusable engineering capabilities

The roadmap includes a marketplace where third party publishers can offer toolkits, hardware adapters and compiler bridges. An instrumentation specialist could package a family of device integrations, with a simulated counterpart and example scenarios where supported. A domain expert could provide measurement analysis or test functions. A compiler specialist could offer a bridge for a supported execution environment.

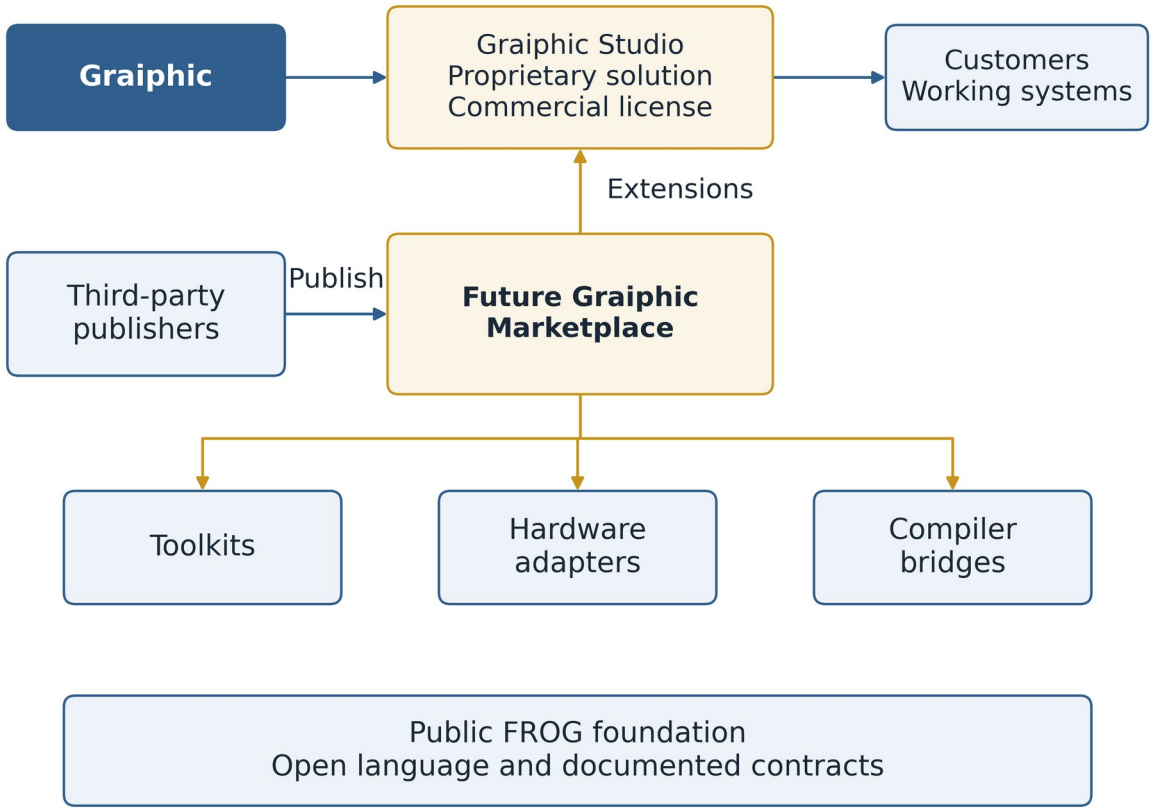


Figure 13 A commercial solution with an open foundation and a growing ecosystem

Graiphic supplies the proprietary Studio product under licence. A future marketplace connects third party publishers with customers through toolkits, adapters and compiler bridges. Commercial details and publication requirements remain to be defined.

The intended model combines licence revenue for Studio with the opportunity to distribute and support ecosystem components. Publisher terms, any distribution fees and revenue sharing arrangements will be defined separately. The value proposition is reusable engineering work, easier discovery and a consistent integration experience.

Trust belongs in the component contract

A marketplace component should explain what it does, which versions and targets it supports, which external capabilities it uses and who maintains it. For a compiler bridge, this includes semantic coverage. For an instrument adapter, it includes the supported device and driver configurations.

Evidence, version history and support commitments help customers choose an appropriate component. Some components may carry additional qualification or service guarantees under specific terms. The marketplace should make those distinctions understandable rather than treating every package as interchangeable.

This is an economic opportunity for Graiphic and for specialist publishers. It also creates a practical benefit for customers: less repeated integration work and a growing set of capabilities that remain compatible with the inspection model.

14 Results and guarantees that can be demonstrated

Measure the complete engineering task

We want Graiphic Studio to produce gains in comprehension, experimentation, debugging, change review, integration and deployment. These outcomes should be measured on tasks that reflect real use.

A useful evaluation compares complete workflows with the same requirements and acceptance criteria. It records the experience of participants, available documentation, agent configuration and target environment. It includes the time needed to inspect and repair the result, rather than stopping when code has been generated.

Result sought	A useful measure	Evidence needed
Faster comprehension	Time and accuracy in explaining a dependency or effect.	Defined questions, comparable programs and recorded answers.
Earlier defect discovery	Defects found before full bench integration and effort needed to reproduce them.	Defined scenarios, confirmed defects and a comparable baseline workflow.
Shorter debugging cycles	Time to locate, correct and verify a defect.	Known task conditions and a confirmed correction.
More effective review	Detection of consequential changes and incorrect assumptions.	Changes with established expected findings.
Integration savings	Engineering effort to connect and validate a supported device.	Device, driver, target and acceptance conditions.
Deployment confidence	Correspondence between accepted source and built artefact.	Versioned build inputs and traceable outputs.

These measures define our validation programme. They will turn the product thesis into comparable results and guide investment toward the capabilities that create the greatest engineering value. Each release should strengthen the connection between an improved experience and a demonstrated gain.

Guarantees need a property and a scope

A meaningful guarantee states what is guaranteed, under which conditions and through which evidence or commitment. Examples include a supported compatibility contract, a traceable relationship between accepted source and build inputs, or a defined service commitment under a commercial agreement.

Our direction is to make such guarantees precise enough to be useful in an engineering decision. The presence of an AI agent or a graphical view cannot guarantee the absence of every defect. The strength of the solution should come from explicit semantics, appropriate checks and evidence that supports the claims made for a release.

The roadmap follows the product thesis

The priorities are to deepen dataflow inspection and runtime understanding, develop controlled scenarios and simulated interfaces, strengthen the connection from source to execution, expand compiler bridges beyond the initial LLVM proof, and demonstrate value on working systems.

Simulation should progress through useful, explicit capabilities. Controlled inputs and reusable scenarios establish a foundation. Simulated adapters extend it to external interactions. Physical models and additional execution modes can then expand the scope where the necessary contracts and evidence exist. Release documentation should identify the supported level, observation capabilities and remaining target validation for each configuration.

Language imports should grow through supported subsets and clear editing contracts. Instrumentation toolkits should establish repeatable integration benefits. The marketplace should make these capabilities easier to discover, obtain and maintain.

This progression connects innovation to results. Each step should improve the engineer's ability to understand the software and deliver a useful system. The economic model can then reward the value created by Graiphic and by the wider ecosystem.

Appendix C collects the proposed development gates, resource questions and acceptance tests in the Heilmeier framework. It provides a common basis for discussing the programme with industrial partners, researchers and investors while keeping delivery commitments tied to a defined scope.

Conclusion

We believe AI is opening a new era of industrial software engineering. As machines contribute more implementation, effective human control becomes a strategic industrial capability.

The graphical dataflow programming language gives that control a practical foundation. Dependencies are explicit, scenarios have defined interfaces, and runtime observations connect to meaningful operations. Highlighting, probes and breakpoints help explain what happened. Actionable edits and compiler bridges carry the engineer's decision into the next result.

This continuity is the moat Graiphic is building through faithful representations, effective human interaction, reliable compilation paths and reusable integrations. Research and industrial evaluation must establish the gains and define the guarantees that make the solution useful.

Graiphic Studio brings these capabilities into a proprietary, general purpose solution, with a concrete industrial application in acquisition, electronic instrumentation and test systems. Its commercial licence and future marketplace provide a model for sustained innovation, specialist participation and reusable engineering value.

We want faster software creation to lead to faster understanding, stronger human control and better working systems. That is the direction of Graiphic.

Appendix A Terminology

Term	Meaning in this whitepaper
FROG	The open graphical dataflow programming language and its documented program representations.
Graiphic Studio	The proprietary development solution developed by Graiphic.
Dataflow	A programming model in which explicit dependencies and the availability of required data contribute to execution eligibility.
Runtime highlighting	A visual connection between the graph and observations of an execution.
Simulation	Execution under controlled inputs, substituted interfaces or explicit models to explore behaviour within a defined scope.
Simulated adapter	A substitute for a supported external interface, with defined responses, state, timing and error behaviour.
Scenario	A defined set of conditions, inputs and expected outcomes used to examine program behaviour.
Probe	An observation point used to inspect a value and its execution context.
Human in the Loop	A workflow in which a person can understand, influence and accept a specific result.
Trustworthy AI	The engineering objective of making AI contributions inspectable, governable and supported by appropriate evidence.
FIR	FROG Intermediate Representation, used in the language and compilation architecture.
Compiler bridge	An integration that maps supported program semantics to a compiler or execution environment under a defined contract.
Language import bridge	An integration that maps a supported source language and build context into an inspection or executable representation.
MCP	Model Context Protocol, used to connect AI applications to tools and contextual resources.
Marketplace	A planned distribution channel for third party toolkits, adapters and compiler bridges.
Moat	A defensible advantage accumulated through technical depth, product quality, reliable integrations and ecosystem value.

Appendix B How to read the Orville chart

The chart is a strategic framework designed for this whitepaper. Its areas describe families of development approaches, and its Graiphic Studio marker represents a target architecture.

The horizontal axis describes the integration of AI into software production: how directly agents can contribute to creating, revising and managing the program representation.

The vertical axis describes the continuity of human control from dataflow to runtime: how effectively the workflow connects program structure, detailed inspection, simulation, debugging, correction and the subsequent build. A high position requires useful links between these activities, together with clear coverage and configuration boundaries.

The positions are qualitative. They are not derived from a benchmark, market share, price comparison or numerical scoring model. Overlapping regions acknowledge that products can combine capabilities and move as they develop. The chart does not establish exclusive ownership of the upper right area.

A future empirical comparison should define supported tasks, products and versions, then evaluate comprehension accuracy, scenario based defect discovery, correction time, observation coverage and the connection between accepted source and executable results. This would turn the positioning thesis into a testable product comparison.

Appendix C The Heilmeier case for Graiphic Studio

This appendix applies the eight questions of the Heilmeier framework to Graiphic's programme. [31] It connects the industrial thesis to the proposed research, resources and evidence. The answers describe our direction and evaluation plan; they do not announce a funded programme, a delivery calendar or measured product results.

1 The objective in plain language

We want engineers to understand and improve software even when they did not write its first implementation. Graiphic Studio should let them follow the relevant logic visually, try defined situations, examine what happened during execution and carry a correction into the next working version.

The industrial objective is to shorten the path from a generated proposal to a system that people can accept and maintain. The scope includes human authored FROG, agent generated FROG and supported language imports. An executable graphical view must retain a validated relationship to the program being built. Human responsibility requires enough understanding and control to make that acceptance meaningful.

2 Current practice and the remaining limitation

Engineers already combine coding agents, source review, graphical environments, debuggers, tests and instrument tools. Established solutions offer dataflow execution, runtime inspection, model to code tracing and AI assistance. Chapters 3, 11 and 12 describe relevant examples. Our starting point recognises those capabilities.

The remaining opportunity is to reduce the effort of connecting them across a complete task. A reviewer may need to reconstruct dependencies from source, relate a result to a particular build, reproduce the relevant conditions and ensure that a correction reaches the accepted executable. Our hypothesis is that a shared representation and dependable mappings can reduce that effort as the volume of generated software grows.

3 The innovation and the basis for confidence

Graiphic proposes a coherent workflow around graphical dataflow: generation, hierarchical inspection, controlled scenarios, runtime observations, correction and compilation. Its novelty lies in the integration and the engineering contracts that keep these activities connected. JSON and dataflow are established building blocks; they do not alone constitute the moat.

The public FROG foundation and initial LLVM compiler bridge provide the starting point. [2] Established dataflow and model based engineering techniques also show that explicit representations can support inspection and execution tracing. [3][23] This makes the direction technically grounded, while leaving substantial research and product work. Chapter 12 identifies the central questions: semantic preservation, abstraction at scale, execution mappings, simulation boundaries and improved human judgment.

4 The people who benefit and the potential impact

Acquisition and test engineers, instrumentation specialists, automation teams, equipment integrators and research organisations provide a concrete initial field. The wider architecture is general purpose. NI, Beckhoff, Keysight, ADLINK, UEI, Gantner and Rohde & Schwarz illustrate the varied ecosystem in which supported integrations could create value; their inclusion is not a claim that every device is supported.

Successful results could mean earlier commissioning, shorter fault investigations, reusable integrations and better transfer of engineering knowledge. At organisational scale, the opportunity is to increase the amount of software change that teams can understand and accept. Graiphic's licensed solution and planned marketplace are intended to turn repeatable customer value into a sustainable business. Chapter 13 distinguishes released engineering capacity, cash savings and the costs of adoption.

5 The principal risks and how to investigate them

The technical risk is loss of meaning: an import, abstraction or compiler mapping may hide a consequential effect or alter behaviour. The human risk is false confidence in a clear looking graph. The operational risk is incomplete observation or a simulation that does not represent the conditions that matter on the target. Integration breadth also creates maintenance costs that may exceed the value delivered.

The proposed response is to define coverage contracts, preserve traceability, compare supported transformations against reference behaviour, and evaluate people on tasks with known expected findings. Simulation tests need explicit models and target validation boundaries. Industrial pilots must include migration, training and maintenance effort. Expansion should follow evidence that the workflow remains understandable, technically dependable and economically useful.

6 The resources and programme cost

A consolidated programme budget is not established in this first edition. The cost of the research and development programme must be estimated separately from future Studio pricing.

The main work packages are language and compiler engineering; the inspection and editing experience; runtime instrumentation; simulation and instrument adapters; validation infrastructure; and industrial evaluation and support. A scoped stage needs named responsibilities, estimated engineering effort, hardware and infrastructure allowances, partner participation and provision for uncertainty. Costs depend strongly on the selected language subsets, targets, devices and acceptance requirements.

The next planning deliverable is a costed plan for the bounded demonstrator, followed by a separately scoped pilot. This approach makes the investment decision concrete without treating universal language or hardware coverage as a single deliverable.

7 The sequence and duration

This edition does not establish a delivery calendar. We propose three development gates, with duration estimated once scope, staffing and evaluation access are agreed. These gates are a programme structure, not announced product release dates.

The first gate is a bounded demonstrator linking accepted FROG, graphical inspection, observable execution and a verified correction through a selected compiler path. The second is an industrial pilot with selected equipment, controlled scenarios and comparison against a defined existing workflow. The third expands language, compiler and integration coverage where the earlier evidence supports further investment.

Imports and simulation should grow through explicit supported capabilities. A marketplace follows the component contracts, maintenance responsibilities and user value needed to make third party participation useful. The programme should revise scope when a gate reveals unresolved semantic, usability or economic problems.

8 Intermediate and final tests of success

Before evaluation, each stage should specify its supported configurations, comparison workflow, test corpus, participant experience and acceptance thresholds. Numerical targets remain to be agreed for that scope; they must be fixed before results are collected.

At the intermediate gate, the required semantic cases should pass their defined checks, unsupported constructs should be identified, and observations should point to the relevant accepted version and operations. A graphical correction must reach the subsequent build and satisfy the associated regression cases. Agreement on a finite corpus supports claims about that corpus and contract, not universal correctness.

At the industrial gate, compare complete tasks using comprehension accuracy, confirmed defect detection, time to correct and verify, integration effort, observation overhead and total cost. Include incorrect conclusions and confidence in them, as well as successful outcomes. Faster completion should be accepted as a gain only when the required correctness and coverage criteria are met.

The industrial pilot is the final test of the initial scope. Every later coverage expansion should repeat these acceptance tests for its newly supported configurations.

A successful stage produces reproducible evidence, documented limits and a supported path to deployment. An unsuccessful stage should identify whether the representation, interaction, integration or economic assumption needs revision. This is how the programme can convert an ambitious industrial thesis into results, useful guarantees and a defensible technological advantage.

Appendix D References

The FROG foundation whitepaper is available in the GO-Whitepapers collection. The remaining references are public primary sources consulted on 20 and 21 October 2026. Vendor documentation establishes the cited interfaces and capabilities; it does not establish availability or qualification within Graiphic Studio. Product direction and strategic forecasts are Graiphic's position.

[1] Graiphic. FROG — Opening the Next Era of Graphical System Programming. Foundation whitepaper, version 1.2, April 2026.

[2] Graiphic. FROG public repository and language overview.

[3] NI. Why Dataflow Programming Languages are Ideal for Programming Parallel Hardware.

[4] NI. Debugging Techniques. LabVIEW User Manual, updated 28 September 2026.

[5] OpenAI. Introducing Structured Outputs in the API.

[6] NI. Software Configuration Management and LabVIEW.

[7] NI. LabVIEW VI Scripting.

[8] NI. Nigel AI for Test.

[9] Jan Göbel. G-AI LabVIEW AI Assistant.

[10] Zühlke. labview-mcp public project.

[11] NI. NI-DAQmx Python documentation.

[12] ADLINK. Data Acquisition Software and Utilities.

[13] UEI. Frequently Asked Questions.

[14] Gantner Instruments. GInsData Library.

[15] Beckhoff. Linking C++ ADS library for TwinCAT 3 in Visual Studio.

[16] Beckhoff. Third Party Libraries.

[17] Keysight. Creating IO Applications.

[18] Keysight. Which Type of Instrument Driver Should I Use?.

[19] Rohde & Schwarz. Remote Control and Instrument Drivers.

[20] LLVM. LLVM Language Reference Manual.

[21] Clang. JSON Compilation Database Format Specification.

[22] MLIR. Rationale.

[23] MathWorks. Verify Generated Code by Using Code Tracing.

[24] MathWorks. Simulink Code Inspector.

[25] MathWorks. Simulink Copilot.

[26] Anthropic. Code Review is now generally available.

[27] NI. Connecting a Model Context Protocol Server to NI Nigel AI.

[28] Zühlke. LabVIEW MCP installation and client setup.

[29] Rohde & Schwarz. RsInstrument public project.

[30] MathWorks. SIL and PIL Simulations.

[31] DARPA. The Heilmeier Catechism. The eight question framework is adapted in the opening overview and Appendix C.

[32] DORA. State of AI assisted Software Development 2025. Research overview.

[33] Graiphic. Graiphic Studio development interface screenshots supplied for this whitepaper, 24 October 2026. The application build displayed in the captures is distinct from the whitepaper edition number.